

AI WEATHERWISE

Phase 5: Project Development

Setup, backend implementation and database configuration

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	5 of 8 – Project Development
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Overview

This phase documents how the AI WeatherWise backend was implemented from an empty folder to a running REST API. It follows the epics defined in the planning phase: project setup, backend development and database configuration.

2. Project Setup

2.1 Create the project

1. Create a new folder named **AI Weatherwise**.
2. Open the folder in Visual Studio Code.
3. Open the integrated terminal and initialize the project:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

Install Nodemon for development:

```
npm install --save-dev nodemon
```

Package	Purpose
express	Web framework and routing
mongoose	MongoDB object modeling (ODM)
bcryptjs	Password hashing
jsonwebtoken	Create and verify JWT tokens
cors	Cross-origin resource sharing
dotenv	Load environment variables from <code>.env</code>
multer	Multipart / file upload handling

Package	Purpose
@google/genai	Google Gemini AI integration
nodemon (dev)	Restart server automatically on file changes

2.2 Create the base files and folder structure

Create `index.js`, `.env`, `package.json` and `.gitignore` in the root, and a `src` folder containing `controllers`, `middleware`, `models`, `routes` and `utils`.

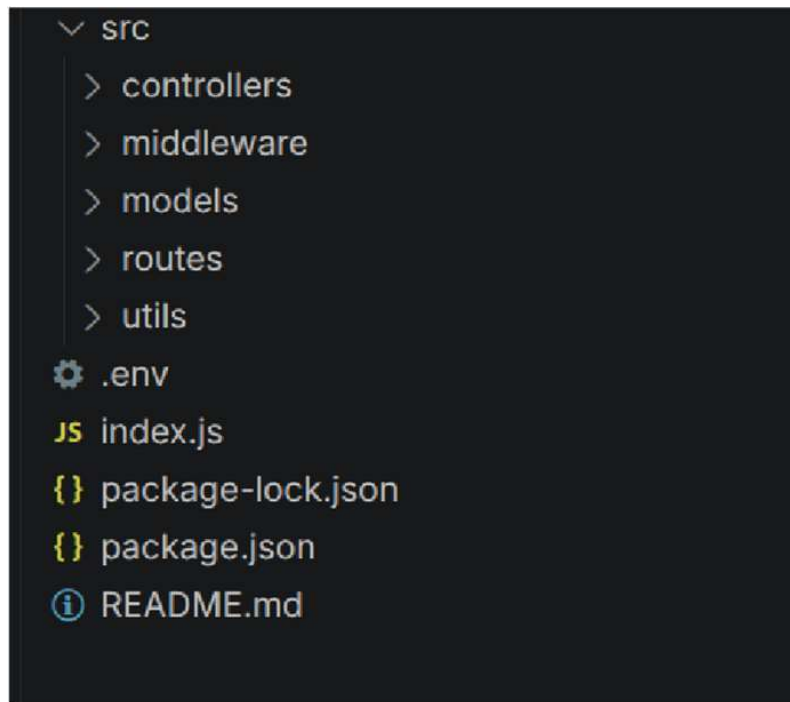


Figure 1: Initial folder structure created in VS Code

2.3 Environment configuration

Create a `.env` file in the project root:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

Security tip: never commit the `.env` file. Add it to `.gitignore` so secrets are not published to the public GitHub repository.

3. Backend Structure

The backend is modular. Routing, controllers, middleware, models, services and configuration are separated to improve maintainability and scalability.

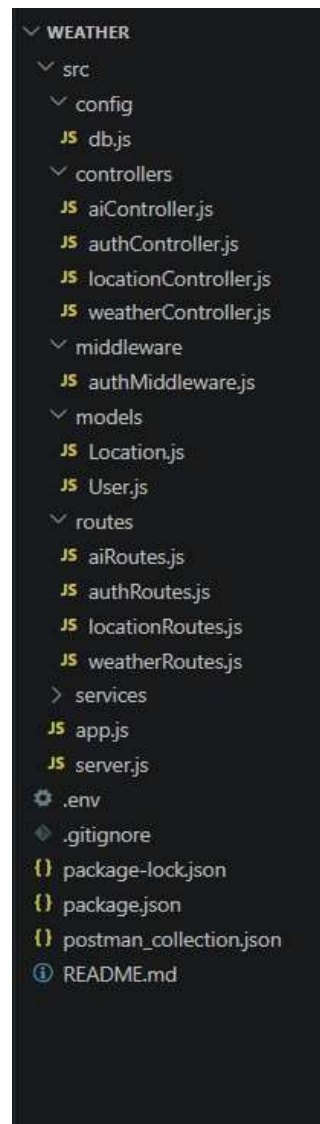


Figure 2: Final backend project structure

Module	Files	Responsibility
config	db.js	Creates the MongoDB connection.
controllers	authController, locationController, weatherController, aiController	Handle requests and build JSON responses.
middleware	authMiddleware	Verifies the JWT and protects private routes.
models	User.js, Location.js	Mongoose schemas and validation.
routes	authRoutes, locationRoutes, weatherRoutes, aiRoutes	Map URL paths to controllers.
services	Weather and AI services	Business logic and external API calls with fallback mode.
app.js / server.js	—	Express configuration and server start-up.

4. Core Implementation

4.1 Application entry point

The entry point loads the Express app, connects to MongoDB and starts listening **only after** the connection succeeds. If the database connection fails, the process logs the error and exits.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

Figure 3: Entry point that starts the server after the database connects

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

4.2 Database connection (config/db.js)

This module connects the application to MongoDB using Mongoose. It uses `MONGO_URI` from the environment and falls back to a local database named `weatherwise`.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Figure 4: MongoDB connection module

```
const mongoose = require('mongoose');
```

```
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI ||
'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

5. Mongoose Schemas

5.1 Location schema

Each location belongs to a user. A compound unique index guarantees that a user cannot save the same city and country twice.

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

Figure 5: Location schema with unique user + city + country index

5.2 User schema

The user schema validates name, e-mail (unique, lowercase, pattern-checked) and password (minimum six characters). The model uses `bcryptjs` so passwords are stored only as hashes, and timestamps are enabled.

```

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\/w+([\.-]?)\/w+@\/w+([\.-]?)\/w+([\.-]?)\/w{2,3}+$/i,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});

```

Figure 6: User schema with validation rules

6. Authentication, Weather and AI Modules

Module	How it works
Registration	Validates input, hashes the password with bcryptjs, saves the user and returns <code>_id</code> , <code>name</code> , <code>email</code> and a JWT (HTTP 201).
Login	Finds the user by e-mail, compares the password with the stored hash and returns user data and a JWT (HTTP 200).
Auth middleware	Reads the token from the request header, verifies it with <code>JWT_SECRET</code> and attaches the user to the request.
Location controller	Creates, lists, updates and deletes favorite cities that belong to the logged-in user.
Weather controller / service	Fetches current weather for a city from OpenWeatherMap. Without an API key it returns mock data flagged with <code>isMock: true</code> .
AI controller / service	Sends temperature, humidity and condition to Google Gemini and returns a recommendation. A fallback message is returned if no key is available.

7. Running the Backend

1. Make sure MongoDB is running and `.env` is configured.
2. Install dependencies:

```
npm install
```

1. Start the server:

```
npm start
```

After successful execution the server connects to MongoDB and listens for API requests. The APIs can then be tested with Postman or Thunder Client (see the Testing phase).

8. Development Notes

- The default port in code is **5000**; the Postman screenshots use **5001**, so `PORT=5001` was set in `.env` during testing.
- The Gemini package appears as `@google/genai` in the install command and as `@google/generative-ai` in the requirements list; keep the package name used in the service code consistent.
- Passwords must never be logged or returned in API responses.

The next phase, **Project Testing**, verifies the implemented APIs.